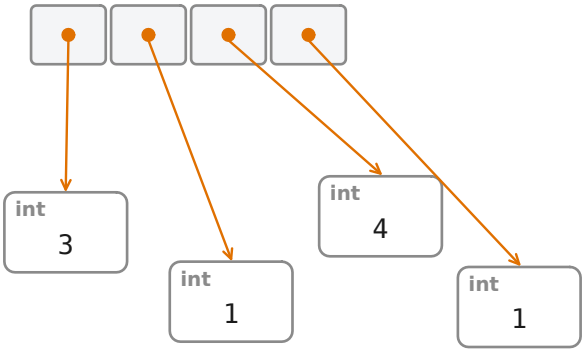
Chapter 18

NumPy done properly

the array is a block of memory with a shape painted on it

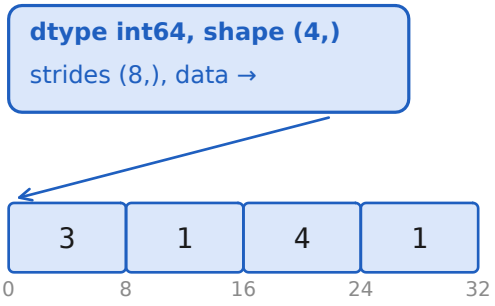
The Computing module showed you NumPy's **what**. This chapter is the **why**: once you can picture an array in memory, views, broadcasting and the speed all stop being magic. Start with the difference that causes everything else:

Python list [3, 1, 4, 1]



4 pointers + 4 full objects, anywhere

np.array([3, 1, 4, 1])



one block: 4 × 8 bytes, no pointers

```
1  import timeit
2  import numpy as np
3
4  days = list(range(1_000_000))
5  arr = np.arange(1_000_000)
6
7  def with_list():
8      return [min(d * 0.25, 5.0) for d in days]      # a million boxed objects, one at a time
9
10 def with_numpy():
11     return np.minimum(arr * 0.25, 5.0)              # C loops over raw doubles
12
13 assert with_list() == with_numpy().tolist()
14 t_list, t_np = (min(timeit.repeat(fn, number=1, repeat=3)) for fn in (with_list, with_numpy))
15 print(f"list {t_list * 1000:.0f} ms, numpy {t_np * 1000:.1f} ms: {t_list / t_np:.0f}x faster")
```

```
list 40 ms, numpy 2.2 ms: 18x faster
```

line 13: same answer first

line 14: best of 3 runs each

Python's loop handles a **boxed object** on every step: look up its type, dispatch the operation, allocate a new object for the result. NumPy checks the type once for the whole block, then runs a tight loop in C over raw machine numbers, often using SIMD instructions to do several at a time.

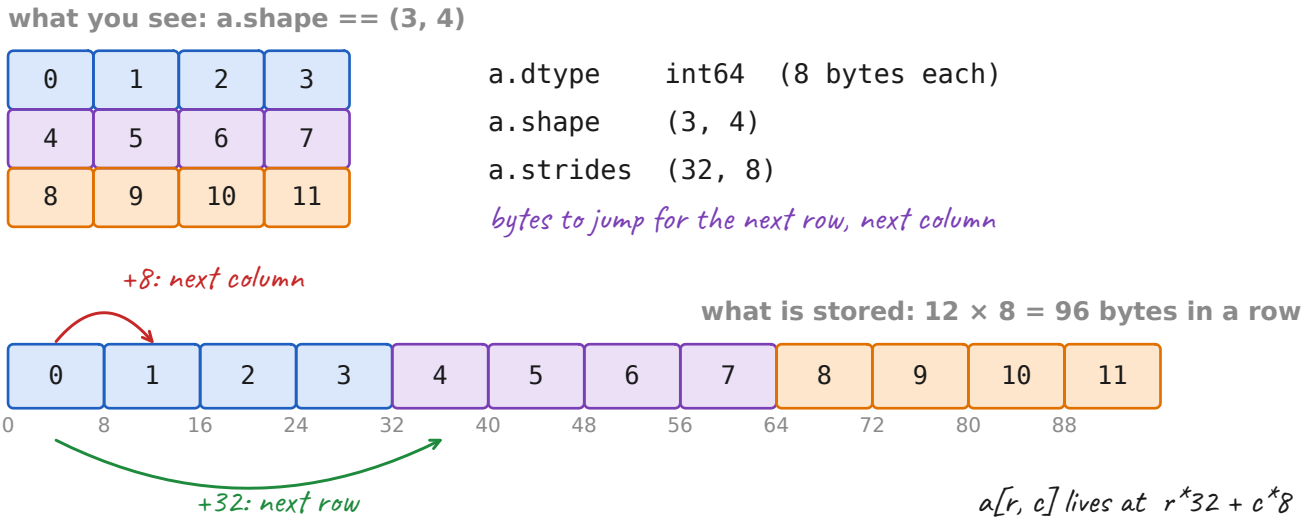
dtype, shape, strides: the three facts

Every array is a **header** plus a **buffer**. The header says how to read the bytes: what each item is (`dtype`), the grid it forms (`shape`), and how many bytes to jump along each axis (`strides`):

```
1 a = np.arange(12).reshape(3, 4)
2 print(a)
3 print(a.dtype, a.shape, a.strides, a.itemsize, a.nbytes)
4 print(a[1, 2], a[1], a[:, 2])
```

← row 1, column 2 / a row / a column

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
int64 (3, 4) (32, 8) 8 96
6 [4 5 6 7] [ 2  6 10]
```



The dtype is a contract: every item has the same type and size, which is what makes the C loop possible. The price is that NumPy numbers behave like C numbers, not Python ones:

```
1 small = np.array([120, 127], dtype=np.int8)
2 print(small + 10)                                  # wraps around, no error
3 print(np.array([1, 2]) / 2, np.array([1, 2]) // 2)
4 print(np.array([1, 2.5, True]).dtype, np.array(["a", 1]).dtype)
5 print(np.array([2**62]) * 4, 2**62 * 4)
```

```
[-126 -119]
[0.5 1. ] [0 1]
float64 <U21
[0] 18446744073709551616
```

line 2: int8 holds -128..127

line 4: everything upcast to one type

line 5: int64 overflow vs Python's big int

dtype	bytes	holds	typical use
`int64` / `int32`	8 / 4	whole numbers, fixed range	counts, indexes (default int)
`uint8`	1	0..255	image pixels
`float64` / `float32`	8 / 4	IEEE floats (Ch 11)	default float / GPUs, big data
`bool`	1	True / False	masks (later this chapter)
`<U10`	40	fixed-width text	labels (pandas usually better)
`object`	8	pointers to Python objects	**avoid**: back to slow boxes

Views and copies: who owns the bytes?

Because the header and buffer are separate, NumPy can make a **new header over the same buffer**. Slicing does exactly that, so it's instant even on a gigabyte array, and writing through the slice writes into the original:

```
1  a = np.arange(12).reshape(3, 4)
2  col = a[:, 1]                          # a view: new header, same bytes
3  col[:] = -1
4  print(a)
5  print(np.shares_memory(a, col), col.strides)
6
7  picked = a[[0, 2]]                     # fancy indexing: always a copy
8  picked[:] = 99
9  print(a[0], np.shares_memory(a, picked))
10
11 t = a.T                                # transpose: just swap the strides
12 print(t.shape, t.strides, np.shares_memory(a, t))
```

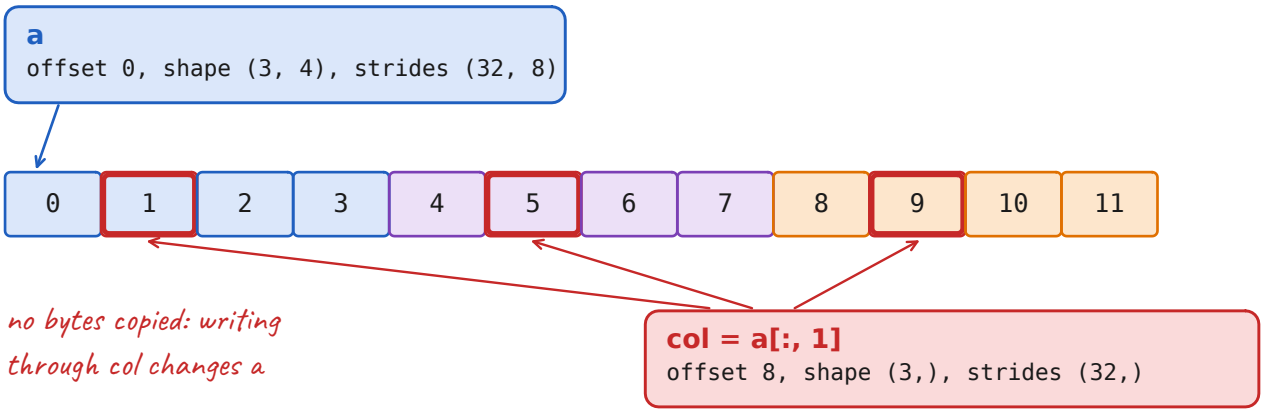
```
[[ 0 -1  2  3]
 [ 4 -1  6  7]
 [ 8 -1 10 11]]
True (32,)
[ 0 -1  2  3] False
(4, 3) (8, 32) True
```

line 3: writes into a

line 11: no data moved at all

operation	result	why
`a[1:3]`, `a[:, 1]`, `a[::2]`	view	a regular step: offset + strides describe it
`a.reshape(...)`, `a.T`, `a.ravel()`	view (usually)	same bytes, new shape or strides
`a[[0, 2]]`, `a[a > 3]`	copy	irregular picks can't be described by strides
`a * 2`, `np.sqrt(a)`, `a.copy()`	new array	its own memory, independent of `a`

Here is the first view in memory: two headers reading the same twelve numbers.



TRAP

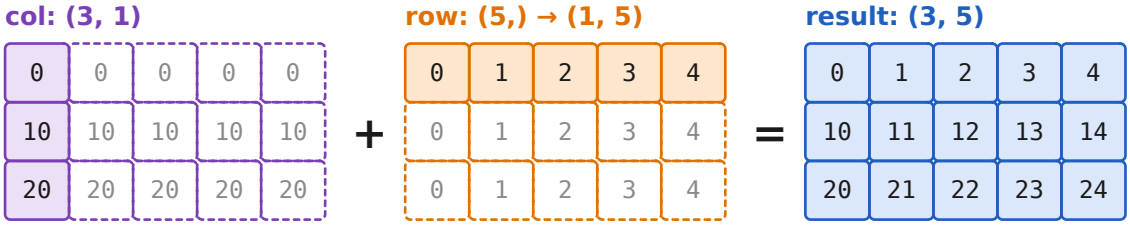
A function that receives a slice and modifies it in place is modifying the caller's array. If a function shouldn't have side effects, `.copy()` its input or build a new result. Strides also explain speed: walking along the last axis reads neighbouring bytes (cache-friendly); walking down a column of a big array jumps through memory.

Broadcasting: shapes that stretch

Arithmetic between arrays of different shapes works if NumPy can stretch them to a common shape. The rule is short enough to memorise:

RULE OF THUMB

Line the shapes up **from the right**. Each pair of sizes must be **equal**, or one of them must be **1** (a missing axis counts as 1). Axes of size 1 are stretched to match.



dashed cells are never stored: NumPy re-reads the same value (a stride of 0)

```python
days = np.arange(0, 25, 6)                      # 0, 6, 12, 18, 24 days late: shape (5,)
rates = np.array([[0.10], [0.25], [0.50]])      # one rate per row: shape (3, 1)
print(np.minimum(days * rates, 5.0))            # (3, 1) with (5,) -> a (3, 5) fee table

due = np.array([3, 10, 4])
print(due[:, None] - due[None, :])              # every pair: (3, 1) - (1, 3) -> (3, 3)
```

```
[[0.  0.6 1.2 1.8 2.4]
 [0.  1.5 3.  4.5 5. ]
 [0.  3.  5.  5.  5. ]]
[[ 0 -7 -1]
 [ 7  0  6]
 [ 1 -6  0]]
```

line 3: no loops, no copies of days or rates

line 6: None adds an axis of size 1

```python
np.ones(3) + np.ones(4)
```

```
ValueError: operands could not be broadcast together with shapes (3,) (4,)
```

shapes	line up from the right	result
`(3, 4)` and `(4,)`	4 = 4; 3 vs missing (1)	`(3, 4)`: the row is added to every row
`(3, 4)` and `(3,)`	4 vs 3: neither is 1	**error**: use `(3, 1)` for one value per row
`(3, 1)` and `(1, 5)`	1 vs 5, 3 vs 1	`(3, 5)`: an outer table
`(2, 3, 4)` and `(3, 1)`	4 vs 1, 3 = 3, 2 vs missing	`(2, 3, 4)`

Vectorise: masks instead of if, whole arrays instead of for

A comparison on an array gives a **boolean array** of the same shape: a mask. Masks replace the `if` inside a loop; `np.where` is the vectorised `x if cond else y`:

```
1 rng = np.random.default_rng(7)
2 days_late = rng.integers(-10, 30, size=12)
3 print(days_late)
4
5 late = days_late > 0                              # bool array, same shape
6 print(late.sum(), days_late[late])                # True counts as 1; a mask selects
7 fees = np.where(late, np.minimum(days_late * 0.25, 5.0), 0.0)
8 print(fees.sum(), fees.max(), fees.argmax())
```

```
[27 15 17 25 13 21 23 -1 -8  2  1 24]
10 [27 15 17 25 13 21 23  2  1 24]
37.0 5.0 0
```

line 5: a mask

line 7: where(cond, if_true, if_false)

line 8: argmax: the index of the max

a Python loop

```
total = 0.0
for d in days_late:
    if d > 0:
        total += min(d * 0.25, 5.0)
print(total)
```

```
37.0
```

vectorised

```
paid = np.clip(days_late, 0, None) * 0.25
total = np.minimum(paid, 5.0).sum()
print(total)
```

```
37.0
```

reads like the formula ■■■■■

speed on big arrays ■■■■■

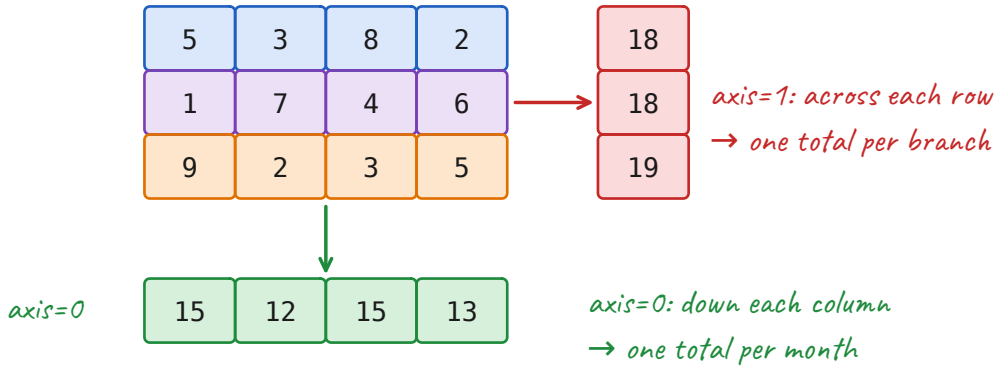
Same answer. The vectorised version states **what** to compute over the whole array and leaves the looping to C, which is where the speed on the chapter's first page came from.

TRAP

`if days_late > 0:` raises `ValueError: The truth value of an array ... is ambiguous`: an array of booleans isn't one boolean. Say which you mean with `.any()` or `.all()`. Likewise combine masks with `&`, `|`, `~` and parentheses, `(a > 0) & (a < 10)`, never `and` / `or`.

Axes and reductions

A **reduction** (`sum`, `mean`, `max`, `argmax`, `std`...) collapses an axis. `axis=0` runs down the rows and leaves one value per column; `axis=1` runs across and leaves one value per row. Here: loans at 3 branches (rows) over 4 months (columns):




```
1 loans = np.array([[5, 3, 8, 2],
2                   [1, 7, 4, 6],
3                   [9, 2, 3, 5]])
4 print(loans.sum(axis=0), loans.sum(axis=1), loans.sum())
5 print(loans.argmax(axis=1))                          # busiest month per branch
6 share = loans / loans.sum(axis=1, keepdims=True)     # (3, 4) / (3, 1)
7 print(share.round(2))
```

```
[15 12 15 13] [18 18 19] 55
[2 1 0]
[[0.28 0.17 0.44 0.11]
 [0.06 0.39 0.22 0.33]
 [0.47 0.11 0.16 0.26]]
```

line 6: keepdims leaves (3, 1), ready to broadcast

HONEST NOTE

Floating-point sums lose precision as they grow (Ch 11). `np.sum` uses pairwise summation, which is far more accurate than a naive loop, and `float64` is the safe default; `float32` accumulates error quickly. For an exact sum of Python floats there's `math.fsum`.

Random numbers, linear algebra, a plot

Make randomness **reproducible** with a seeded `Generator` (not the old `np.random.seed` global). Then let linear algebra fit a straight line through noisy data by least squares:

```
rng = np.random.default_rng(seed=2026)
copies = rng.integers(1, 6, size=40)                # copies of each title on the shelf
loans = 3 * copies + rng.normal(0, 2, size=40)      # the hidden rule, plus noise

A = np.column_stack([copies, np.ones(40)])          # rows: [copies, 1]
(slope, intercept), *_ = np.linalg.lstsq(A, loans)
print(f"loans ≈ {slope:.2f} × copies + {intercept:.2f}")

M, v = np.array([[2, 1], [1, 3]]), np.array([5, 10])
x = np.linalg.solve(M, v)                           # solve M @ x == v exactly
print(x, M @ x)
```

```
loans ≈ 2.77 × copies + 1.49
[1. 3.] [ 5. 10.]
```

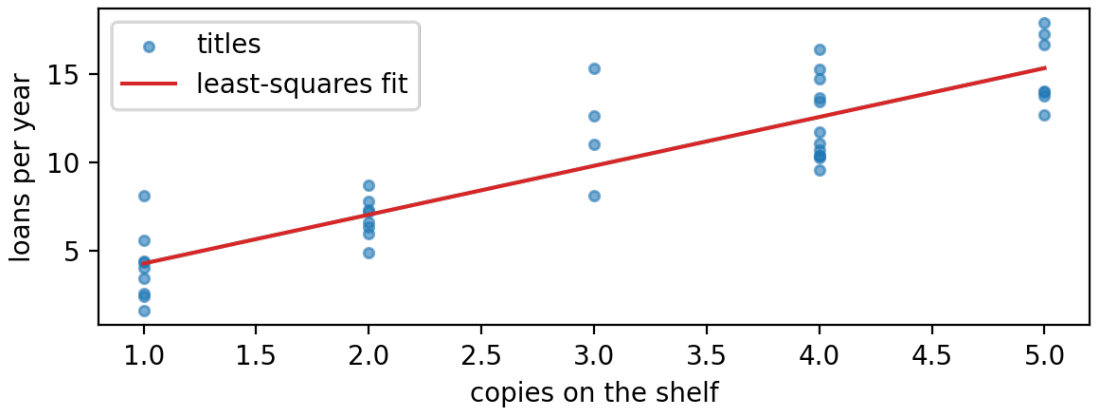
line 1: same seed, same numbers, every run

line 6: best slope and intercept

line 10: @ is matrix multiplication

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(6, 2.4))
ax.scatter(copies, loans, s=14, alpha=0.6, label="titles")
xs = np.array([1, 5])
ax.plot(xs, slope * xs + intercept, color="C3", label="least-squares fit")
ax.set(xlabel="copies on the shelf", ylabel="loans per year")
ax.legend()
fig.tight_layout()                          # then plt.show() or fig.savefig("fit.png")
```



pandas, briefly: labelled columns on top of NumPy

pandas adds names: a DataFrame is a table whose columns are NumPy arrays with labels, missing-value handling and SQL-like operations (Ch 12's GROUP BY in one line):

```
1 import pandas as pd
2
3 df = pd.DataFrame({"branch": ["north", "south", "north", "east", "south", "north"],
4                    "title": ["Dune", "Emma", "Emma", "Dune", "Dune", "Ulysses"],
5                    "days": [14, 21, 7, 14, 28, 14]})
6 print(df.groupby("branch")["days"].agg(["count", "mean"]))
7 print(type(df["days"].to_numpy()).__name__, df["days"].to_numpy().dtype)
```

```
        count       mean
branch
east        1  14.000000
north       3  11.666667
south       2  24.500000
ndarray int64
```

line 6: GROUP BY branch

line 7: underneath: an ndarray

reach for	when
plain Python + csv/sqlite3	small data, mixed types, or logic per record
NumPy	numbers in a grid, the same operation on every element
pandas	tables with named, differently-typed columns: filter, group, join
Matplotlib	any plot; pandas' .plot() calls it for you

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 18.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 18.1` shows other valid solutions and their trade-offs.

18.1 build from a spec A fee table by broadcasting
18.2 fix the bug normalise_rows must not touch its input
18.3 build from a spec A moving average without loops
18.4 learn an unseen module Group without a loop: unique and bincount
18.5 measure and explain Nearest branch for 20,000 readers
18.6 mini-project Game of Life on a torus

PASSPORT STAMP ask the same question of every new language (Appendix B)

2. Memory model

An `ndarray` is one contiguous typed buffer plus a header (dtype, shape, strides). Slices are views that share the buffer; fancy and boolean indexing copy.

6. Data structures

`ndarray` (n-dimensional, one dtype), pandas `Series`/`DataFrame` (labelled columns).

12. Stdlib and tooling

Not stdlib: `numpy`, `matplotlib`, `pandas` from pip (Ch 19). The stdlib's `array` module and `statistics` cover small cases.

IN OTHER LANGUAGES

C: an array **is** this buffer, and a 2-D array is row-major exactly like NumPy's default. Fortran, MATLAB and Julia store columns first. R and MATLAB are vectorised by design. Java has primitive arrays but no broadcasting. Rust: the `ndarray` crate. GPUs (PyTorch, JAX) use the same shape-and-strides model.

DEEP DIVE (your vault)

Vault notes: 16.5 Why an array abstraction exists · 16.6 Dtypes are representation contracts · 16.7 Shape, strides, and physical layout · 16.8 Views, copies, and aliasing · 16.9 Broadcasting as shape algebra · 16.10 Vectorisation and native kernels · 16.11 Reductions, axes, and numerical stability · 16.12 Randomness, generators, and reproducibility · 16.13 Linear algebra as array structure · 16.18 Grouping, aggregation, and transformation · 16.25 Visualisation is an encoded argument.